

¿Qué son los estados en React? ¿Cuáles son sus características y para qué sirve?
Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

El estado es la forma que tiene React de guardar datos que cambian en el tiempo por la interacción del usuario con el sistema.

Características:

- Estado mutable, a diferencia de los props.
- Es lo que le da a React la cualidad de ser interactivo, el usuario interactúa con la UI y eso cambia un estado, lo cual permite modificar la apariencia de la UI en respuesta.
- Visto en código, son una array de JavaScript de dos elementos: el valor actual de la variable que representa al estado y una función que determina el cambio de valor del estado.

Ejemplo:

```
function Contador() {
  // acá vemos que el state es un array
  // se llama al Hook useState
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Cantidad de clics: {count}</p>

      {/* definimos que la funcion de cambio de estado cambia el estado
      incrementando el valor actual de la variable */}
      <button onClick={() => setCount(count + 1)}>Sumar</button>
    </div>
  );
}
```

¿Qué son los eventos en React? ¿Cuáles son sus características y para qué sirve?
Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo

Un evento son las acciones que el usuario realiza sobre la UI y a las cuales nuestro código React debe responder.

Sirven para ejecutar lógica de JavaScript cuando el usuario interactúa con la interfaz. En JSX, los atributos de los eventos se escriben siempre en formato camelCase (por ejemplo onClick en lugar del onclick de HTML).

Un evento es por ejemplo un clic del usuario en algún botón de la UI que cumple alguna funcionalidad particular. Siguiendo con el ejemplo anterior, vemos cómo se maneja la acción de que el usuario haga clic en el botón “Sumar” utilizando el evento onClick y pasándole una función anónima que realiza un cambio de estado (incrementa el valor).

Otros eventos de React son onChange (se dispara cada vez que el valor de un campo de entrada cambia) y onSubmit (se asigna directamente a la etiqueta <form> y reacciona cuando el usuario envía el formulario).

**¿Qué son los Hook en React? ¿Cuáles son sus características y para qué sirve?
¿Cuáles son los principales Hook? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.**

Los Hooks son funciones propias de React que permiten darle a los componentes (las funciones que crean los elementos que renderizamos en pantalla) funcionalidades complejas necesarias para que la interfaz sea interactiva (como la existencia de estados con el hook useState) y reaccione de determinada forma en momentos específicos (efectos con useEffect).

Los principales Hook son los mencionados:

- useState: permite agregar, inicializar y manejar variables de estado dentro del componente.
- useEffect: es el Hook que permite implementar efectos secundarios en un componente. Normalmente, el trabajo de un componente de React es calcular y devolver JSX (la interfaz visual). Todo lo que ocurra fuera de ese cálculo visual se considera un efecto secundario (por ejemplo consultar bases de datos, cambiar el título de la pestaña, etc).

El uso de useEffect en un componente significa que luego de que se dibuje la pantalla (que es trabajo del componente) se ejecutará una acción de fondo.

Recibe dos argumentos: una función flecha con la acción que se quiere ejecutar y, opcionalmente, un arreglo de dependencias [] que determina cuándo debe dispararse esa función.

El comportamiento de useEffect depende de la lista de dependencias. Si está vacía, la lógica del efecto secundario se ejecuta una sola vez cuando el componente se renderiza por primera vez. Si en el arreglo se indican variables, se ejecuta la primera vez que carga la pantalla y, a partir de ahí, únicamente si cambia el valor de alguna de las variables indicadas.

Ejemplo:

```
import { useEffect } from 'react';

export function ListaProductos() {
  // le pasamos la función flecha y el arreglo de dependencias vacío
  useEffect(() => {
    //para el ejemplo solo simulamos una búsqueda en base de datos, ese es el
    //efecto secundario cuando se renderice la lista de productos
    console.log("Buscando productos en la base de datos...");
  },
  // le pasamos tmb arreglo vacío indicando que solo se tiene que "buscar en la
  // base de datos" la primera vez que se quiere renderizar la lista de prodcutos
  []);

  return <h1>Cargando catálogo...</h1>;
}
```